

بسمه تعالی

پروژه درس برنامه نویسی پیشرفته

دانشکده فنی انقلاب اسلامی

دانشگاه ملی مهارت

استاد درس: دکتر اردستانی

نام و نام خانوادگی دانشجو: امیر پارسا علیزاده

شماره دانشجویی: 02220040709015

نکته: در تمام این پروژه ها a آخرین رقم غیر صفر شماره دانشجویی شماست.

گزارش تحلیل کد: شبیه‌سازی بازی تعادل پاندول معکوس (Inverted Pendulum)

۱. معرفی کلی پروژه

این برنامه شبیه‌سازی یک مسئله‌ی کلاسیک در حوزه‌ی مکانیک و کنترل (Control Theory) به نام "پاندول معکوس روی گاری (Cart-Pole / Inverted Pendulum)" است. کاربر با فشردن کلیدهای چپ/راست به گاری نیرو وارد می‌کند تا از سقوط پاندول (که در حالت تعادل ناپایدار قرار دارد) جلوگیری کند.

۲. دسته‌بندی بخش‌های کد

بخش	کلاس/محدوده	نقش
مدل داده‌ی فیزیکی	Pendulum, Cart	نگهداری متغیرهای حالت (State Variables)
ورودی کاربر	Controller	تبدیل کلید فشرده‌شده به نیروی اعمالی
هسته‌ی محاسباتی	Simulation	حل معادلات دیفرانسیل حرکت و تشخیص باخت
رابط کاربری و حلقه‌ی اجرا	GUI	رسم، حلقه‌ی زمان‌بندی‌شده، مدیریت وضعیت بازی

۳. تحلیل کلاس‌های مدل فیزیکی

۳.۱. Cart و Pendulum

این دو کلاس صرفاً به عنوان ساختار داده (Data Class) عمل می‌کنند و متغیرهای حالت سیستم فیزیکی (زاویه، سرعت زاویه‌ای، موقعیت، سرعت خطی) را نگه می‌دارند. جداسازی این دو موجودیت از یکدیگر، با اینکه در واقعیت به هم متصل‌اند (پاندول روی گاری سوار است)، امکان می‌دهد معادلات فیزیکی در کلاس Simulation به صورت مستقل از نمایش گرافیکی نوشته شوند.

۳.۲. کلاس Controller

```
FORCE_MAGNITUDE = 12.0
def apply_left(self):
    self.manual_force = -self.FORCE_MAGNITUDE
```

این کلاس یک لایه‌ی میانی بین رویدادهای صفحه‌کلید و مقدار عددی نیرو ایجاد می‌کند. این انتزاع (Abstraction) باعث می‌شود در آینده بتوان منبع نیرو را بدون تغییر در Simulation عوض کرد (مثلاً جایگزینی با یک الگوریتم کنترل خودکار به جای صفحه‌کلید).

۴. تحلیل کلاس Simulation — مهم‌ترین و پیچیده‌ترین بخش کد

۴.۱. معادلات حرکت (Equations of Motion)

```
temp = (force + m * l * theta_dot ** 2 * math.sin(theta)) / (M + m)
theta_acc = (g * math.sin(theta) - math.cos(theta) * temp) / (
    l * (4.0 / 3.0 - (m * math.cos(theta) ** 2) / (M + m))
)
x_acc = temp - (m * l * theta_acc * math.cos(theta)) / (M + m)
```

۴.۲. روش انتگرال‌گیری عددی Euler Method :

```
self.cart.velocity += x_acc * dt
self.cart.x += self.cart.velocity * dt
self.pendulum.angular_velocity += theta_acc * dt
self.pendulum.angle += self.pendulum.angular_velocity * dt
```

این بخش از روش اوایلر صریح (Explicit/Forward Euler Method) برای حل عددی معادلات دیفرانسیل استفاده می‌کند: ابتدا شتاب محاسبه شده و سرعت را به‌روزرسانی می‌کند، سپس سرعت (به‌روزشده) موقعیت را به‌روزرسانی می‌کند. این تکنیک با نام **Semi-implicit Euler** یا **Euler-Cromer** نیز شناخته می‌شود که نسبت به روش اوایلر ساده، برای سیستم‌های نوسانی پایداری عددی بهتری دارد (چون از سرعت جدید به‌جای سرعت قدیم برای به‌روزرسانی موقعیت استفاده می‌کند).

دقت این روش وابسته به کوچک بودن گام زمانی ($dt=0.02$) است؛ در گام‌های بزرگ‌تر ممکن است خطای انباشتی (Accumulated Error) در شبیه‌سازی‌های طولانی‌مدت مشهود شود. روش‌های دقیق‌تر مانند Runge-Kutta مرتبه ۴ می‌توانستند دقت بالاتری ارائه دهند، هرچند برای یک بازی تعاملی real-time، اوایلر ساده انتخاب معقولی از نظر کارایی محاسباتی است.

۴.۳. اصطکاک مصنوعی

```
self.cart.velocity *= self.CART_FRICTION # 0.999
```

این خط یک میرایی مصنوعی (Artificial Damping) به سیستم اضافه می‌کند که در معادلات فیزیکی اصلی وجود نداشت. از نظر تحلیلی، این تکنیک اغلب برای شبیه‌سازی‌های بازی به کار می‌رود تا از انباشت خطای عددی و رفتار غیرواقعی جلوگیری شود.

۴.۴. شرط پایان بازی

```
if abs(self.pendulum.angle) > self.ANGLE_LIMIT:
    self.game_over = True
if abs(self.cart.x) > self.CART_LIMIT:
    self.game_over = True
```

تشخیص باخت بر پایه‌ی دو محدودیت فیزیکی مستقل است: زاویه‌ی بحرانی (۵۰ درجه) و محدوده‌ی حرکت گاری. این دقیقاً معیار پایان‌دهی است.

۵. تحلیل کلاس GUI

۵.۱. حلقه‌ی اجرا (Game Loop) با after

```
def _loop(self):
    if not self.simulation.game_over and not self.paused:
        self.simulation.step(self.controller.manual_force, self.DT)
    ...
    self._draw()
    self.root.after(20, self._loop)
```

اینجا هم از الگوی **Non-blocking Game Loop** با `root.after()` استفاده شده تا رابط کاربری در حین اجرای پیوسته‌ی شبیه‌سازی مسدود نشود. تابع `step` مقدار ثابت $DT = 0.02$ را دریافت می‌کند، در حالی که فاصله‌ی واقعی فراخوانی `after(20, ...)` نیز ۲۰ میلی‌ثانیه است؛ یعنی گام زمانی شبیه‌سازی با گام زمانی واقعی هماهنگ شده است — (Fixed Timestep) این نکته‌ی مهمی است چون در صورت عدم تطابق، شبیه‌سازی نسبت به سرعت واقعی کند یا تند می‌شد.

۵.۲. تبدیل مختصات فیزیکی به پیکسل

```
cart_px = self.CENTER_PX + self.cart.x * self.PIXELS_PER_METER
...
bob_x = pivot_x + length_px * math.sin(self.pendulum.angle)
bob_y = pivot_y - length_px * math.cos(self.pendulum.angle)
```

این بخش یک تبدیل مختصات (**Coordinate Transformation**) بین فضای فیزیکی (متر، رادیان) و فضای گرافیکی (پیکسل) است. موقعیت گوی پاندول با استفاده از روابط مثلثاتی استاندارد محاسبه می‌شود.

۵.۳. مدیریت رویدادهای صفحه‌کلید

```
self.root.bind("<KeyPress-Left>", lambda e: self.controller.apply_left())
self.root.bind("<KeyRelease-Left>", lambda e: self.controller.stop())
```

تفکیک بین `KeyPress` و `KeyRelease` امکان کنترل پیوسته (Continuous Control) را فراهم می‌کند: نیرو تا زمانی که کلید نگه‌داشته شده اعمال می‌شود و با رها شدن، متوقف می‌شود؛ برخلاف رویکرد ساده‌تری که فقط با هر بار فشردن یک ضربه‌ی گسسته (Discrete Impulse) وارد کند.